

Chapter 9: Optimization

Gregory S. DeLozier, PhD

SQL describes the result we want. The database chooses how to find it. Optimization starts with understanding that choice: which rows must it read, and what work can an index save?

We will begin with the seven-row `FruitsForSale` table from SQLite's query planning documentation. (1) Fruit, state, and price are enough to see how a scan differs from an indexed search, why column order matters, and how an index can help with sorting. Then we will create a titles database from IMDb, the Internet Movie Database, and measure those same ideas on millions of records. (2)(3)

The fruit examples below are explained through the illustrations. The hands-on work begins with the titles database, using the files in `chapter-09-optimization/code`.

About the Illustrations

The fruit, state, and price illustrations in this chapter come from SQLite's "Query Planning" article. (1) SQLite dedicates its documentation to the public domain, which permits us to reproduce these illustrations. (4)

The original illustrations and explanations are at <https://www.sqlite.org/queryplanner.html>. The related articles "EXPLAIN QUERY PLAN" (5) and "The SQLite Query Optimizer Overview" (6) are at <https://www.sqlite.org/eqp.html> and <https://www.sqlite.org/optoverview.html>. SQLite's public-domain statement is at <https://www.sqlite.org/copyright.html>. (4)

Why a Query Is Slow

An ordinary SQLite rowid table has a unique integer key called the rowid. A column declared `integer primary key` names that key. The fruit table leaves it hidden and has three visible columns: (1)

```
create table fruitsforsale (  
    fruit text,  
    state text,  
    price real  
);
```

Each row records a fruit, the state where it was grown, and its price. The table can contain oranges from both Florida and California. Figure 1 shows the rows in rowid order.

rowid	fruit	state	price
1	Orange	FL	0.85
2	Apple	NC	0.45
4	Peach	SC	0.60
5	Grape	CA	0.80
18	Lemon	FL	1.25
19	Strawberry	NC	2.45
23	Orange	CA	1.05

Figure 1. Logical layout of the table. From the SQLite query planning documentation. (1)

Suppose you ask for the price of peaches, and no index exists on the fruit:

```
select price from fruitsforsale where fruit = 'Peach';
```

The database has only one thing it can do: read every row, check the fruit, and keep the ones that match. That is a full table scan. (7) With seven rows nobody notices. As the table grows, the scan checks more rows even when only one matches.

rowid	fruit	state	price
1	Orange	FL	0.85
2	Apple	NC	0.45
4	Peach	SC	0.60
5	Grape	CA	0.80
18	Lemon	FL	1.25
19	Strawberry	NC	2.45
23	Orange	CA	1.05

Figure 2. A full table scan. (1)

A lookup by rowid is different. The rows are stored in rowid order, so SQLite can use a binary search: look at the middle, decide which half holds the answer, and repeat. (8) A binary search over N rows takes about $\log N$ steps, where a scan takes about N . Computer scientists write those as $O(\log N)$ and $O(N)$, a way of describing how the work grows as the data grows. (9) For a table of 10 million rows, the SQLite documentation estimates that finding a row by binary search is roughly a million times faster than scanning all the rows. (1)

rowid	fruit	state	price
1	Orange	FL	0.85
2	Apple	NC	0.45
4	Peach	SC	0.60
5	Grape	CA	0.80
18	Lemon	FL	1.25
19	Strawberry	NC	2.45
23	Orange	CA	1.05

Figure 3. Lookup by rowid. (1)

Nobody wants to know the price of “row 4”, though. You want the peaches. That is what an index is for.

Indexes

An index is a second, sorted copy of one or more columns, stored with the rowid of the row each value came from.

(10) You can think of it as the index at the back of a book: the entries are in order, so you find one quickly, and each entry points to the place where the full content lives. SQLite keeps indexes in a balanced tree structure called a B-tree, which keeps searching fast as the index grows. (11)

```
create index Idx1 on fruitsforsale(fruit);
```

fruit	rowid
Apple	2
Grape	5
Lemon	18
Orange	1
Orange	23
Peach	4
Strawberry	19

Figure 4. An index on the fruit column. The rowid breaks ties between equal fruits. (1)

With the index in place, the peach query takes two steps. A binary search of the index finds “Peach” and its rowid, 4. A second binary search of the table finds row 4 and reads the price.

fruit	rowid
Apple	2
Grape	5
Lemon	18
Orange	1
Orange	23
Peach	4
Strawberry	19

rowid	fruit	state	price
1	Orange	FL	0.85
2	Apple	NC	0.45
4	Peach	SC	0.60
5	Grape	CA	0.80
18	Lemon	FL	1.25
19	Strawberry	NC	2.45
23	Orange	CA	1.05

Figure 5. Indexed lookup for the price of peaches. (1)

When the fruit has several matches, SQLite finds the first entry with one binary search and then moves down the index to the next entry, repeating the table lookup for each. Moving to the next entry is cheap because it is usually on the same page of the file, so the cost is roughly one binary search plus one per matching row. (1)

fruit	rowid
Apple	2
Grape	5
Lemon	18
Orange	1
Orange	23
Peach	4
Strawberry	19

rowid	fruit	state	price
1	Orange	FL	0.85
2	Apple	NC	0.45
4	Peach	SC	0.60
5	Grape	CA	0.80
18	Lemon	FL	1.25
19	Strawberry	NC	2.45
23	Orange	CA	1.05

Figure 6. An indexed lookup that matches several rows. (1)

This is worth remembering for later. **An index makes finding rows cheap. Reading many rows through an index is still work.**

Reading a Query Plan

We can inspect the work SQLite plans to do on the fruit table. Putting `explain query plan` in front of a query returns a description of the plan instead of the answer. (5) For example:

```
explain query plan
select price from fruitsforsale where fruit = 'Peach';
```

The last column of each row is the description. The words to know are these:

Plan text	Meaning
SCAN fruitsforsale	Read every row of the table.
SEARCH fruitsforsale USING INDEX name (column=?)	Use the index to find matching rows, then read them from the table.
SEARCH ... USING COVERING INDEX name	Answer from the index alone, without reading the table.
USE TEMP B-TREE FOR ORDER BY	Collect the rows and sort them in a temporary structure.

The word to look for first is **SCAN**. A scan of a big table is where slow queries come from. A scan of a small table is fine. The plan also changes as the database changes, which is the point of the exercise. The text of the plan is meant for people debugging a query, and SQLite may change its wording between versions, so do not write code that depends on it. (5)

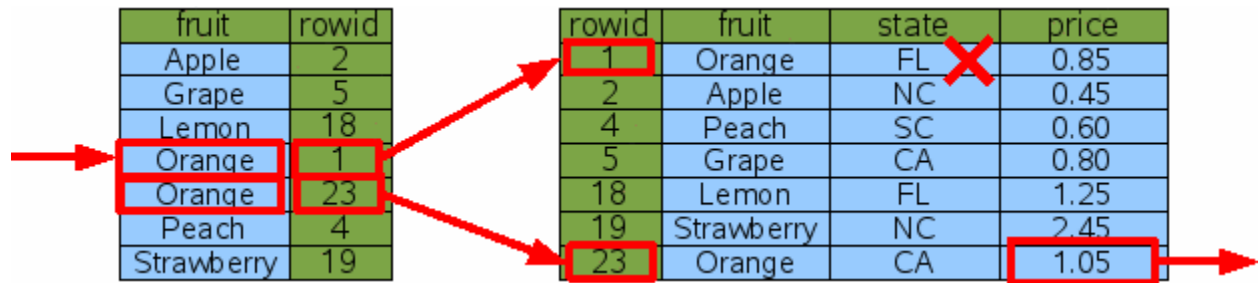
The plan is a query planner's decision. (12) The SQL is the same whichever plan is chosen, and so is the answer. The plans differ only in how much work they do. That is what query optimization means. (13) SQLite estimates the cost of each candidate plan and picks the cheapest one it can see. (1)

Fruit and State Together

Suppose we want the price of California oranges:

```
select price from fruitsforsale
where fruit = 'Orange' and state = 'CA';
```

An index on fruit finds both orange rows. SQLite then reads them and rejects the Florida row. (1)

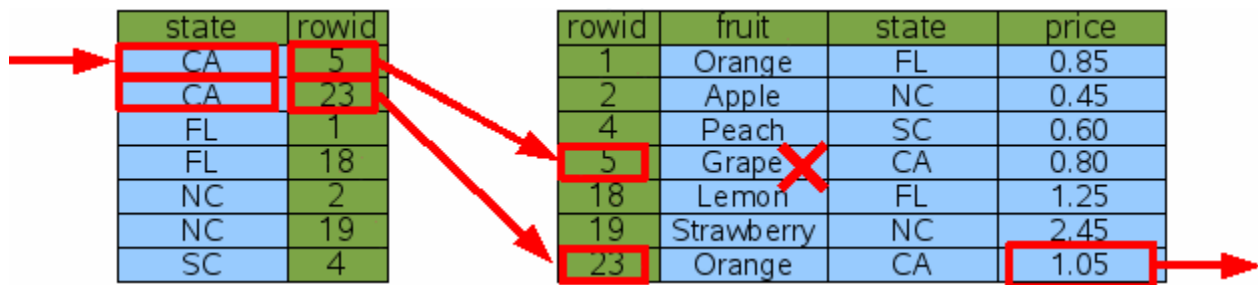


Find oranges first, then check their state. (1)

An index on state offers another route:

```
create index Idx2 on fruitsforsale(state);
```

SQLite can find the California rows and reject any fruit that is not an orange. These two separate indexes do not become one combined AND search. (1)



Find California fruit first, then check the fruit name. (1)

A **multi-column index** puts both conditions in one ordered structure. (14)

```
create index Idx3 on fruitsforsale(fruit, state);
```

fruit	state	rowid
Apple	NC	2
Grape	CA	5
Lemon	FL	18
Orange	CA	23
Orange	FL	1
Peach	SC	4
Strawberry	NC	19

Fruit order, with state breaking ties between equal fruits. (1)

SQLite can locate the California orange entry directly, then use its rowid to read the price.

fruit	state	rowid
Apple	NC	2
Grape	CA	5
Lemon	FL	18
Orange	CA	23
Orange	FL	1
Peach	SC	4
Strawberry	NC	19

rowid	fruit	state	price
1	Orange	FL	0.85
2	Apple	NC	0.45
4	Peach	SC	0.60
5	Grape	CA	0.80
18	Lemon	FL	1.25
19	Strawberry	NC	2.45
23	Orange	CA	1.05

Search using both columns. (1)

The order is fruit first, then state within each fruit. All orange entries are together, so a search on fruit alone can also use this index. California entries are spread among the fruit groups, so state alone does not identify one contiguous range. (1)

fruit	state	rowid
Apple	NC	2
Grape	CA	5
Lemon	FL	18
Orange	CA	23
Orange	FL	1
Peach	SC	4
Strawberry	NC	19

rowid	fruit	state	price
1	Orange	FL	0.85
2	Apple	NC	0.45
4	Peach	SC	0.60
5	Grape	CA	0.80
18	Lemon	FL	1.25
19	Strawberry	NC	2.45
23	Orange	CA	1.05

A search on the first column can use the two-column index. (1)

For these examples, Idx3 can serve the queries that used Idx1. Keeping both would mean maintaining two indexes for the same searches. We will examine column order and the statistics-dependent skip-scan exception on the larger database. (6)

Covering Indexes

Consider the last step of the peach lookup again. After the index search there is a second search into the table to read the price. If the index also carried the price, the second search would be unnecessary. An index that holds every column a query uses is a **covering index**. (1)

```
create index Idx4 on fruitsforsale(fruit, state, price);
```

fruit	state	price	rowid
Apple	NC	0.45	2
Grape	CA	0.80	5
Lemon	FL	1.25	18
Orange	CA	1.05	23
Orange	FL	0.85	1
Peach	SC	0.60	4
Strawberry	NC	2.45	19

Figure 13. A covering index. The price is stored in the index. (1)

fruit	state	price	rowid
Apple	NC	0.45	2
Grape	CA	0.80	5
Lemon	FL	1.25	18
Orange	CA	1.05	23
Orange	FL	0.85	1
Peach	SC	0.60	4
Strawberry	NC	2.45	19

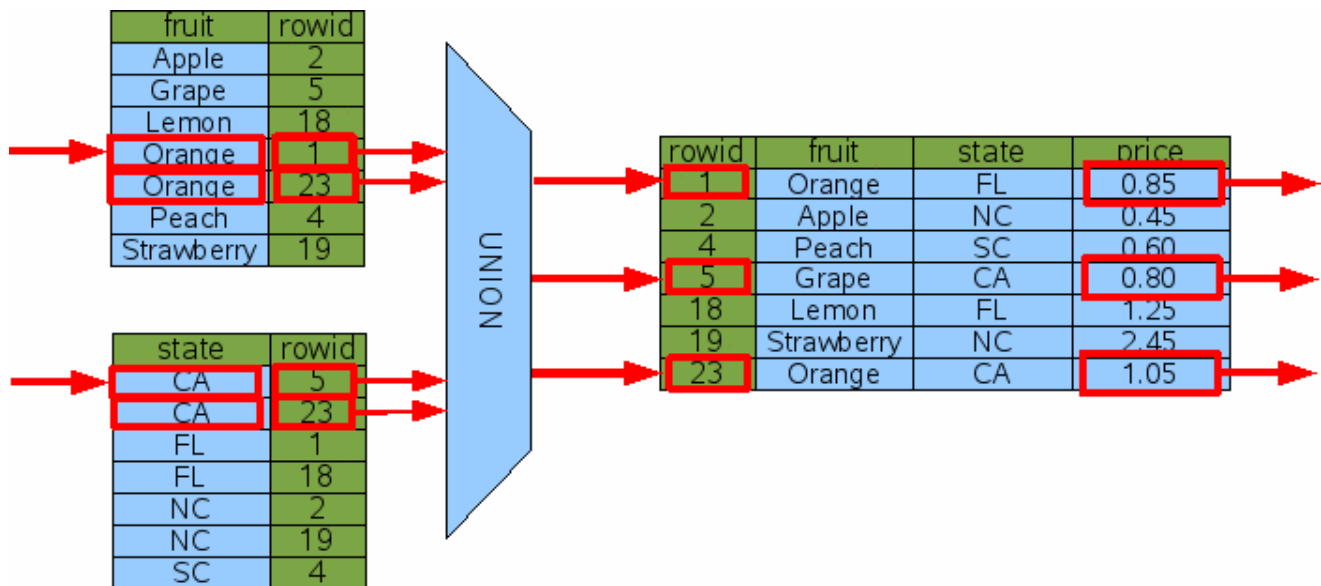
Figure 14. The query never touches the original table. (1)

OR on the Fruit Table

An OR query asks for either condition to match:

```
select price from fruitsforsale
where fruit = 'Orange' or state = 'CA';
```

SQLite can use the fruit index for one term and the state index for the other, then combine the matching rowids without returning the California orange twice. (1)



Separate indexed searches combined for an OR query. (1)

Both terms need a usable index for this strategy. If one requires a full table scan, SQLite may choose a scan for the whole query. The plan tells us which strategy it selected. (1)

Sorting

Sorting has the same structure as searching. Without help, `order by` collects the results and sorts them in a temporary structure. That takes time proportional to $K \log K$ for K rows, and it needs somewhere to hold them all. (1)



Figure 16. Sorting without an index. (1)

SQLite avoids sorting whenever the order comes free. Reading the table in rowid order produces rowid order:

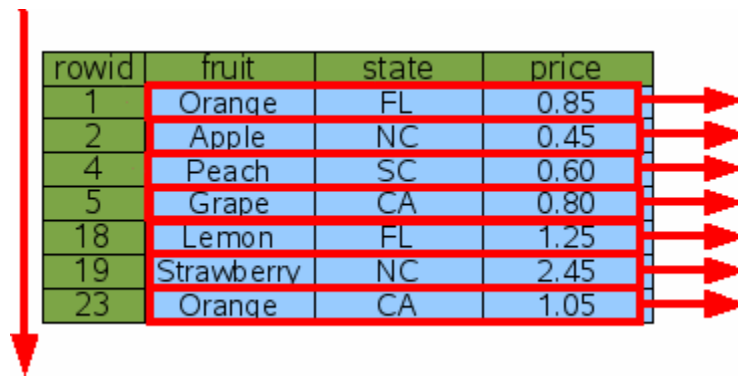


Figure 17. Sorting by rowid. (1)

An index is in order by definition, so reading the index from top to bottom gives the rows in order by the indexed column. Reading it from bottom to top gives descending order. With an ordinary index, each entry still needs a lookup in the table, which costs about as much as the sort it replaced, but it does not need temporary storage and it can start returning rows immediately.

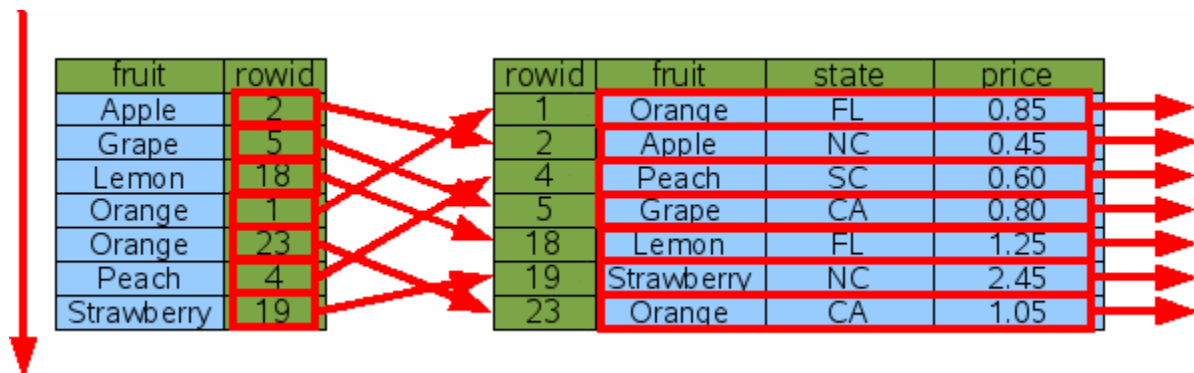
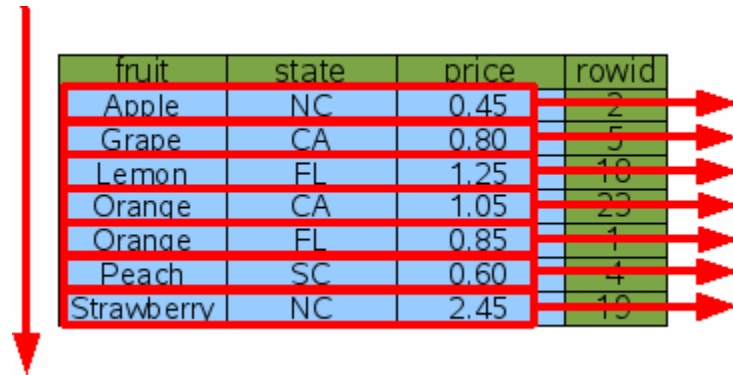


Figure 18. Sorting with an index. (1)

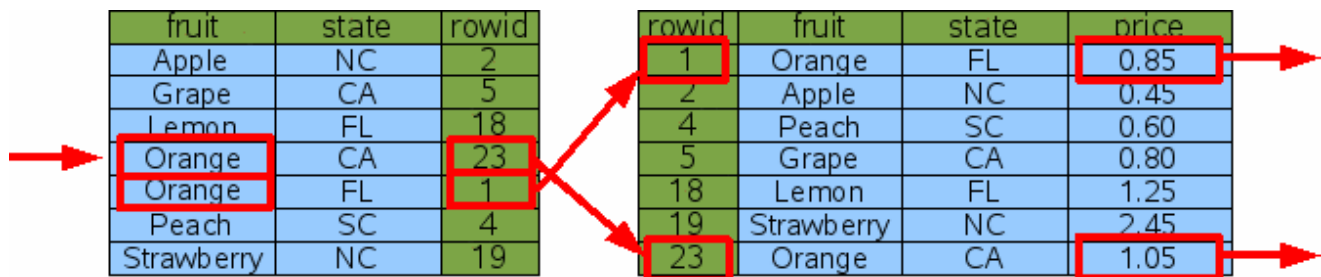
With a covering index the lookups disappear, and the scan is proportional to N with no buffer:



fruit	state	price	rowid
Apple	NC	0.45	2
Grape	CA	0.80	5
Lemon	FL	1.25	18
Orange	CA	1.05	23
Orange	FL	0.85	1
Peach	SC	0.60	4
Strawberry	NC	2.45	19

Figure 19. Sorting with a covering index. (1)

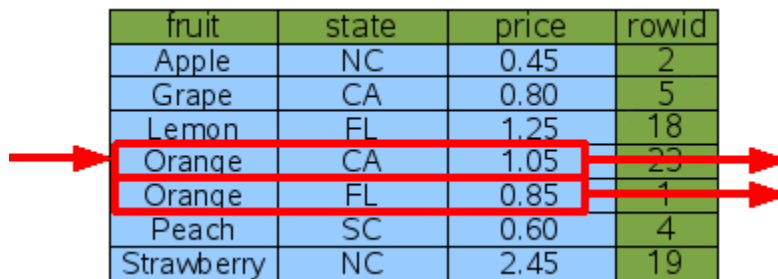
Search and sort can share one index. With an index on (fruit, state), the query for oranges ordered by state needs no sort, because all the orange entries are together and already in state order:



fruit	state	rowid
Apple	NC	2
Grape	CA	5
Lemon	FL	18
Orange	CA	23
Orange	FL	1
Peach	SC	4
Strawberry	NC	19

rowid	fruit	state	price
1	Orange	FL	0.85
2	Apple	NC	0.45
4	Peach	SC	0.60
5	Grape	CA	0.80
18	Lemon	FL	1.25
19	Strawberry	NC	2.45
23	Orange	CA	1.05

Figure 20. Search and sort with one multi-column index. (1)



fruit	state	price	rowid
Apple	NC	0.45	2
Grape	CA	0.80	5
Lemon	FL	1.25	18
Orange	CA	1.05	23
Orange	FL	0.85	1
Peach	SC	0.60	4
Strawberry	NC	2.45	19

Figure 21. Search and sort with a covering index. (1)

Sometimes an index gets only part of the way. For order by fruit, price, the index delivers rows in order by fruit, and SQLite sorts each group of equal fruits separately. Many small sorts use less memory and can begin returning rows sooner than one large sort. (1) In the plan this appears as USE TEMP B-TREE FOR LAST TERM OF ORDER BY.



Figure 22. A partial sort. (1)

Telling SQLite About Your Data

When several indexes could serve a query, SQLite chooses using estimates of how selective each one is: how many rows an index lookup is likely to return. The analyze command gathers those statistics and stores them in the database. (15) Without them the choice between similar indexes is close to arbitrary. (1) Run it after loading a lot of data and after building indexes. Later, we will see statistics enable a skip-scan on the titles database:

```
analyze;
```

Getting the Data

The small examples let us follow every row. Now we will build imdb.db and measure the cost of the same operations on a larger table. The snapshot used here has 12,811,750 titles and 1,713,431 ratings. (2)(3)

IMDb publishes tab-separated values (TSV) files, one record per line with the fields separated by tab characters. (16) The files are compressed with gzip, use UTF-8 (a standard text encoding), and write a missing value as \N. They are refreshed daily. IMDb makes them available for personal and non-commercial use, so download them for your own coursework, do not put them in a public repository, and do not redistribute them. (3)

The course supplies import_imdb.py and schema.sql, not a prebuilt copy of imdb.db. IMDb supplies the source records. The Python program does not download them: it reads the two files you obtain below and builds your own SQLite database.

Use Python 3.10 or newer. The loader and SQLite experiments use only Python's standard library, so there are no packages to install. From the root of your course checkout, open the code folder and check Python:

```
cd chapter-09-optimization/code
python3 --version
python3 -m unittest -v
```

The tests use small generated files and temporary databases. They need no download. On a system where Python is named `python`, use that command in place of `python3` throughout.

Stay in this folder. The following are shell commands, not Python statements. Create a data directory, download both files, and run the loader:

```
mkdir -p data
curl -fL -o data/title.basics.tsv.gz https://datasets.imdbws.com/title.basics.tsv.gz
curl -fL -o data/title.ratings.tsv.gz https://datasets.imdbws.com/title.ratings.tsv.gz
python3 import_imdb.py
```

The result is `imdb.db` in the code folder, containing the titles and ratings tables. That is the titles database used by the remaining examples. Keep the `.gz` files in `data/`; there is no need to unpack them. The first build refuses to overwrite an existing database. To deliberately rebuild your local copy from the downloads, use `python3 import_imdb.py --replace`.

The source links are also in the IMDb reference at the end of this chapter. A browser download works too: save the files under the exact names shown above in `code/data/`, then run the loader. Allow several gigabytes of free space for the database and the indexes we will add.

The `title.basics` file is about 227 megabytes (MB) compressed and `title.ratings` is about 9 MB, so the first download takes a while. Keep the files compressed. Our loader reads them as they are.

Each file starts with a header line. Here are the first rows of each, with the columns we will use:

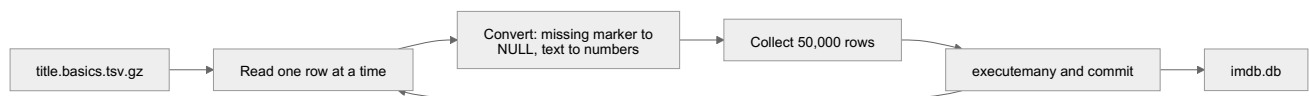
tconst	titleType	primaryTitle	originalTitle	isAdult	startYear	endYear
runtimeMinutes	genres					
tt0000001	short	Carmencita	Carmencita	0	1894	\N
1	Documentary, Short					
tconst	averageRating	numVotes				
tt0000001	5.7	2231				

`tconst` is IMDb's identifier for a title. It is text, not a number, and it is the key that connects a title to its rating. The counts in this chapter come from files downloaded on September 24, 2026, and IMDb refreshes the files daily, so yours will differ slightly. Most of the titles are television episodes:

Title type	Rows
tvEpisode	9,906,741
short	1,158,984
movie	757,581
video	331,493
tvSeries	305,702
all other types	351,249

Loading Data in Chunks

A file this size does not fit comfortably in a program's memory if you read it all at once, and there is no need to. The loader `import_imdb.py` reads the compressed file one row at a time and writes the rows to SQLite in chunks of 50,000 with `executemany`. (17) Memory use stays small no matter how big the file gets, and each chunk is committed, so the database is never waiting on millions of uncommitted rows.



The conversion step matters. IMDb writes a missing value as the two characters `\N`. If we stored those as text, a query such as `where start_year is null` would find nothing, and comparing years as numbers would fail. The loader turns `\N` into a real SQL `NULL`, and it converts the year, runtime, and vote columns to integers and the rating to a real number.

The tables are created from `schema.sql`, which declares each table's columns and a primary key and nothing else. No secondary indexes yet. Loading without them is faster, because the database does not have to maintain them row by row, and it lets us add them later and watch what each one does.

The titles table stores the title information; ratings connects to it through tconst. Here is the schema the loader creates:

```
-- Two of the IMDb files, loaded with no secondary indexes yet.
-- tconst is the text identifier IMDb uses for a title, such as tt0000001.
create table titles (
    tconst text primary key,
    title_type text,
    primary_title text,
    original_title text,
    is_adult integer,
    start_year integer,
    end_year integer,
    runtime_minutes integer,
    genres text
);

create table ratings (
    tconst text primary key,
    average_rating real,
    num_votes integer
);
```

These are ordinary rowid tables. Their text primary keys have automatic indexes, while the rowids remain hidden. The experiments will add secondary indexes on the columns our queries use.

```
python3 import_imdb.py
```

```
Loading title.basics.tsv.gz
titles: 1,000,000 rows
...
titles: 12,811,750 rows in total
53.8 seconds
Loading title.ratings.tsv.gz
ratings: 1,713,431 rows in total
3.8 seconds
```

The whole load took about a minute on the laptop used for this chapter, and the database file came to 1,508 MB. The script refuses to overwrite an existing `imdb.db` unless you add `--replace`, for the same reason `build_mystery.py` did in the dataset topic.

Every timing in this chapter comes from one laptop, one afternoon, and SQLite version 3.53.4 running under Python 3.14. Your numbers will differ. The pattern will not: what matters is whether a query is 10 times faster or 10,000 times faster, not the exact milliseconds (ms).

Check the Database You Built

After the loader finishes, the code folder should contain these files:

```
code/
  import_imdb.py
  schema.sql
  imdb.db
  data/
    title.basics.tsv.gz
    title.ratings.tsv.gz
```

At the shell, enter `python3` to start an interactive session. Paste the following Python code at the `>>>` prompt without typing the prompt itself. A blank line finishes an indented block in this interpreter. Open the existing database read-only for this check. This connection fails if the file is missing, rather than creating an empty database under a mistyped name.

```
from pathlib import Path
import sqlite3

print(Path("imdb.db").resolve())
connection = sqlite3.connect("file:imdb.db?mode=ro", uri=True)
print(connection.execute(
    "select name from sqlite_master where type = 'table' order by name"
).fetchall())
for table in ("titles", "ratings"):
    count = connection.execute(f"select count(*) from {table}").fetchone()[0]
    print(table, count)

print(connection.execute("""
    select t.tconst, t.primary_title, t.start_year,
           r.average_rating, r.num_votes
    from titles t left join ratings r on r.tconst = t.tconst
    where t.tconst = ?
""", ("tt0000001",)).fetchone())
connection.close()
exit()
```

The table list should contain ratings and titles. Both counts should be nonzero and agree with the loader's totals. The example identifier selects *Carmencita*, dated 1894; its rating and vote count may change with the daily files. A left join keeps a title in the result even if it has no rating record.

If the loader reports a missing file, check that both `.tsv.gz` files are in `data/` beneath your current folder. A failed or interrupted download should be downloaded again before importing. An interrupted import can leave a partial `imdb.db`; after correcting the cause, run `python3 import_imdb.py --replace` to rebuild from the beginning. That option discards the existing database and its indexes.

Run and Compare the Experiments

Back at the shell, run:

```
python3 explore_indexes.py
python3 compare_python_sql.py
```

The first script prints a heading for each experiment, the SQL, the plan, and the best of three timings. When it creates an index it also reports build time and added database size. It resets its `idx_` indexes and planner statistics at the beginning, then leaves the new indexes in place. Run it before `compare_python_sql.py`, which compares Python processing with SQL on the same database.

To keep the output for comparison, run `python3 explore_indexes.py > index-results.txt`. Redirection saves the output instead of displaying progress in the terminal, so let the command finish. Record the query, the index used, elapsed time, and index size. Compare the answers as well as the speed.

For a manual experiment, reopen `imdb.db` with a writable connection in Python. Start `python3` at the shell and enter the code below. The plan helper prints the explanation returned by SQLite.

```
import sqlite3
from time import perf_counter

connection = sqlite3.connect("file:imdb.db?mode=rw", uri=True)

def plan(sql, parameters=()):
    for row in connection.execute("explain query plan " + sql, parameters):
        print(row[3])

connection.execute("drop index if exists idx_titles_primary_title")
query = "select tconst, start_year from titles where primary_title = ?"
parameters = ("The Godfather",)
plan(query, parameters)
started = perf_counter()
before = connection.execute(query, parameters).fetchall()
print("Before:", (perf_counter() - started) * 1000, "ms")
connection.execute("create index idx_titles_primary_title on titles(primary_title)")
connection.commit()
plan(query, parameters)
started = perf_counter()
after = connection.execute(query, parameters).fetchall()
print("After:", (perf_counter() - started) * 1000, "ms")
print("Same rows:", sorted(before) == sorted(after))
connection.close()
```

The timer includes fetching every result, not just starting the query. Sorting the two result lists checks their contents without assuming an order the SQL did not request. This one measurement is a first comparison; repeat it because file caching affects the result. The supplied experiment script repeats each timing three times.

Watching an Index Work

The script `explore_indexes.py` runs each experiment in this chapter against `imdb.db`. It prints the query, the plan, and the best of three timings, and it builds each index as it goes. Before it begins it removes any index whose name starts with `idx_`, so you can run it as many times as you like. On the full data it takes several minutes, most of it in scans and index builds.

```
python3 explore_indexes.py
```

The first step is a lookup by the primary key:

```
select primary_title, start_year from titles where tconst = ?
plan: SEARCH titles USING INDEX sqlite_autoindex_titles_1 (tconst=?)
time: 0.0 ms
```

The database made that index for us when we declared the primary key. The time rounds to 0.0 ms, which means under a tenth of a millisecond, out of 12.8 million rows.

Now a search by a column with no index. There are 47 titles named “The Godfather”, so this returns a small list:

```
select tconst, start_year from titles where primary_title = ?
plan: SCAN titles
time: 4,535.8 ms
```

A scan like this took between about 0.7 and 4.7 seconds across our runs, depending on whether the operating system already had the file in memory. On the run shown it took more than four seconds. Add an index:

```
create index idx_titles_primary_title on titles(primary_title);
```

```
built in 18.4 seconds; the database grew by 381 MB
plan: SEARCH titles USING INDEX idx_titles_primary_title (primary_title=?)
time: 0.0 ms
```

Four and a half seconds became under a tenth of a millisecond. The index took 18 seconds to build and added 381 MB, about a quarter of the size of the original table. That is the tradeoff. Indexes make some reads far faster, and they cost disk space and time to build. They also cost time on every later write, because the database must update each index when a row changes. On a scratch table, 500,000 inserts took 0.25 seconds with no index, 0.37 seconds with one, and 1.06 seconds with three. An application that mostly reads can afford many indexes. One that mostly writes has to be more selective.

The SQLite documentation makes the guarantee that matters for testing: indexes should never change the answer, only how quickly it arrives. (1) If adding an index ever changes the rows you get back, something else is wrong.

Two Conditions, Two Columns

Real searches usually have more than one condition. How many movies came out in 1994?

```
select count(*) from titles where title_type = 'movie' and start_year = 1994;
```

With no useful index this scans the table, and on this run it took 4.4 seconds. Suppose we build an index on `start_year` alone. SQLite uses it to find every title from 1994, reads each row from the table, and throws away the ones that are not movies. This is the same find-then-filter process we used for California oranges.

That took 37.7 ms. Better by a factor of more than a hundred, but it does work that a better index avoids. A second index on the other column does not fix it. SQLite can use either index but not both at once, and using a type index instead changes which rows are read and discarded. (1)

The answer is a **multi-column index**: one index that sorts by the first column and, among equal values, by the second. (14)

```
create index idx_titles_type_year on titles(title_type, start_year);
```

Now one binary search lands on the entry for both conditions at once:

```
plan: SEARCH titles USING COVERING INDEX idx_titles_type_year (title_type=? AND
start_year=?)
time: 0.1 ms
```

The order of the columns matters, and it matters in a way that surprises people the first time. The index is sorted by `title_type` first. Entries for a single type are together, and inside that group they are sorted by year. So a query on `title_type` alone can use the index, because all the entries it wants are in one block. By the ordinary rule, a query on `start_year` alone cannot, because the 1994 entries are scattered, one group inside every type.

We can check. After dropping the single-column year index so the choice is forced, and before running `analyze`, the second column alone looks like this:

```
select count(*) from titles where start_year = ?
plan: SCAN titles USING COVERING INDEX idx_titles_type_year
time: 432.0 ms

select count(*) from titles where title_type = ?
plan: SEARCH titles USING COVERING INDEX idx_titles_type_year (title_type=?)
time: 18.2 ms
```

The first query does not search at all, at least not yet. It walks the whole index, which is faster than reading the whole table because the index is smaller, but it is still a scan. The second searches. The rule from the optimizer documentation is that a search can use the columns of an index from the left, with no gaps, and it stops at the first column that has a range condition instead of an equality. (6) For an index on (title_type, start_year, primary_title):

The query's conditions	How much of the index a search can use
title_type = 'movie'	The first column.
title_type = 'movie' and start_year = 1994	The first two columns.
title_type = 'movie' and start_year >= 1990	Both columns, ending at the range.
start_year = 1994 alone	None, unless SQLite chooses a skip-scan after analyze.
title_type = 'movie' and primary_title = 'X'	Only the first column. There is a gap at the second.

The start_year-alone row has one exception, and it depends on statistics. Once analyze has told SQLite that the first column has few distinct values, SQLite can use a skip-scan. It searches the index once for each distinct value of the first column, so start_year = 1994 becomes a handful of searches instead of a walk of the whole index. (6) We tried it. title_type has only 11 distinct values, and after analyze, which took 29 seconds on the full table, the plan changed:

```
select count(*) from titles where start_year = ?
  plan: SEARCH titles USING COVERING INDEX idx_titles_type_year (ANY(title_type) AND
start_year=?)
  time: 1.9 ms
```

That is 432 ms down to 1.9 ms with no new index. The skip-scan pays only when the leading column has few values, so treat it as a bonus and do not design around it. If a query will often search by start_year alone, give it an index that starts with start_year.

This leads to a housekeeping rule. If one index is the beginning of another, the shorter one is redundant, because the longer one can do its job. For these searches, the index on title_type, start_year can also serve queries on title_type alone. Drop it. (1) It also gives the design advice. Put the columns you test with equality first, and any column you test with a range last.

Covering a Titles Query

Now apply the covering-index idea to IMDb, asking for the names of the 1994 movies rather than a count:

```
select primary_title from titles where title_type = ? and start_year = ?
  plan: SEARCH titles USING INDEX idx_titles_type_year (title_type=? AND start_year=?)
  time: 5.0 ms
```

After we add `primary_title` to the end of the index:

```
create index idx_titles_type_year_title on titles(title_type, start_year, primary_title);
```

```
built in 30.5 seconds; the database grew by 536 MB
plan: SEARCH titles USING COVERING INDEX idx_titles_type_year_title (title_type=? AND
start_year=?)
time: 1.4 ms
```

Notice that the earlier count query already said `COVERING INDEX`. `count(*)` needs no column values at all, so the two-column index covered it. A covering index is a property of the query and the index together.

This is a smaller win than the first index, and the SQLite documentation says so plainly. Adding output columns removes one of the two binary searches, a factor of about two, whereas the first index changed the cost by a factor of a million. (1) Here it is 5.0 ms to 1.4 ms, and it cost 536 MB to get there. For most queries nobody will notice. Build a covering index for a query that runs constantly, not for every query.

OR on the Titles Table

One multi-column index helps most directly when conditions are joined by `AND`. `OR` is handled differently: SQLite may optimize each `OR` term separately and combine the matches. Find the titles that came out in 1894 or are called “The Godfather”:

```
select tconst from titles where start_year = ? or primary_title = ?
```

SQLite treats each side of the `OR` separately, finds the matching rowids through an index for each, combines them, and reads the rows. (1)

The catch is that **every** term needs an index. If one side has none, SQLite has to scan the table for that side anyway, and it may as well use the scan for the whole query. That is what happened with only the title index in place:

```
plan: SCAN titles
time: 4,567.5 ms
```

After indexing `start_year` as well:

```
plan: MULTI-INDEX OR
plan: INDEX 1
plan: SEARCH titles USING INDEX idx_titles_year (start_year=?)
plan: INDEX 2
plan: SEARCH titles USING INDEX idx_titles_primary_title (primary_title=?)
time: 0.1 ms
```

Searching Text with LIKE

Search boxes tend to use like, and like is where indexes most often let people down. SQLite can turn like 'The Godfather%', a match on a fixed beginning, into a range search on an index. But it does so only when the index and the comparison agree about case. (6) SQLite's like ignores case by default, and an ordinary index on primary_title sorts case-sensitively. The result is that the index sits unused:

```
select tconst, primary_title from titles where primary_title like ?
plan: SCAN titles
time: 4,668.9 ms
```

The fix is an index declared with a collating sequence, a rule for comparing text, that ignores case: (14)

```
create index idx_titles_primary_title_nocase on titles(primary_title collate nocase);
```

```
plan: SEARCH titles USING INDEX idx_titles_primary_title_nocase (primary_title>? AND
primary_title<?)
time: 0.2 ms
```

The plan now shows a range, primary_title>? AND primary_title<?. That is the prefix search.

One limit remains. SQLite's case-insensitive matching covers only the ASCII letters by default. 'a' like 'A' is true, but an accented lowercase letter does not match its capital. (18) For IMDb titles in other languages, whether an index is used and whether the matches are correct are separate questions. An extension called ICU (International Components for Unicode) adds full Unicode case folding.

One pattern cannot be rescued by an ordinary index. A wildcard at the front, as in like '%Godfather%', matches text that could sort anywhere, so there is no range to search. The plan is a scan, and it took 5.0 seconds here. If the application needs to search for a word anywhere in a title, that is a different problem that needs a different tool, such as SQLite's full-text search extension, FTS5. (19) It is also a reminder to look at the queries an application actually sends. The IMDb workbook's search page, described at the end of the chapter, uses %term% in exactly this way.

The Ten Most-Voted Titles

The pattern that pays off in real applications is a query that asks for the top few by some value. The ten titles with the most votes:

```
select tconst, num_votes from ratings order by num_votes desc limit 10
```

```
plan: SCAN ratings
plan: USE TEMP B-TREE FOR ORDER BY
time: 77.4 ms
```

The database reads all 1.7 million ratings and sorts them to keep ten. With an index on num_votes:

```
create index idx_ratings_votes on ratings(num_votes);
```

```
built in 0.7 seconds; the database grew by 18 MB
plan: SCAN ratings USING INDEX idx_ratings_votes
time: 0.0 ms
```

The plan still says SCAN, but the sort is gone. SQLite reads the index from its high end, stops after ten entries because of the limit, and never touches the other 1.7 million. A scan that stops early is not a full table scan. That is why the word to look for is SCAN together with what comes after it.

Python or SQL?

There is a tempting alternative to all of this. Pull the rows into Python and do the work there. Sometimes that is the right call. Often it is much slower, and the reasons are worth knowing. The script `compare_python_sql.py` measures two cases.

One query per row. Suppose you have the identifiers of the 2,000 most-voted titles and want each title's name. The obvious code asks for each name separately:

```
for tconst in ids:
    row = connection.execute("select primary_title from titles where tconst = ?",
                             (tconst,)).fetchone()
```

That is 2,000 queries. The same information comes from one join:

```
select t.primary_title from ratings r join titles t on t.tconst = r.tconst
order by r.num_votes desc limit 2000
```

```
one query per title: 148.3 ms (2,000 queries)
one join:           10.3 ms (1 query)
```

This is the same pattern as the N+1 problem in the object-relational-mappers topic, and it has the same cure. Here the gap is 14 times. With SQLite running inside your Python process each query is cheap. With a database server across a network, each query also pays for a round trip, and the gap grows.

Fetching everything to count it. Count the movies in each decade, first by fetching the year of every movie into Python and counting there, and then by asking the database with `group by`:

```
fetch every year, count in Python: 262.5 ms (643,991 rows fetched)
GROUP BY in SQL:                   164.2 ms (15 rows fetched)
same answer: True
```

The two are close, and that deserves a plain statement. SQLite lives inside the same process, so moving 643,991 numbers costs little. The SQL version still does its work with an index and returns 15 rows instead of 643,991, and that is the difference that grows with a real server, a slower network, or more data. The point is not that SQL is always dramatically faster. It is that the database can do the reduction next to the data, and you should not assume that moving work to Python is cheaper without measuring.

Which way is better depends on measurement, not on habit. The plan and a timer are the two instruments.

Stored Procedures

There is a third place to do the work: inside the database. A stored procedure is a named routine saved in the database that an application can call. (20) It moves a sequence of statements to the server, so a single call replaces several round trips. The statements run together in one place, and every application that calls the procedure gets the same rule.

SQLite has no stored procedures. It has no `create procedure` statement, and its documentation of the SQL it supports has none. The example below uses PostgreSQL, a database server with a stored procedure language. (21) (22) We change a pet's owner and keep a log of the change, which takes a read, an update, and an insert:

```
create or replace procedure change_owner(pet integer, new_owner text)
language plpgsql
as $$
declare
    previous text;
begin
    select owner into previous from pets where id = pet for update;
    if not found then
        raise exception 'No pet with id %', pet;
    end if;
    update pets set owner = new_owner where id = pet;
    insert into adoption_log (pet_id, old_owner, new_owner)
    values (pet, previous, new_owner);
end;
$$;

call change_owner(1, 'Sam');
```

The file `stored_procedure.sql` contains the complete example, with the two tables it uses and a query that shows the result. It ran without errors on PostgreSQL 16.2, and calling the procedure with an id that does not exist stopped with `No pet with id 99` and left the tables unchanged. The application makes one call, not three, and the logic that changes the owner and writes the log entry lives in one place in the database.

The costs are real. The logic now lives in two places, the application and the database, and each has to be tested and deployed. Procedure languages differ between database systems, so the code is not portable. A change to the procedure is a change to the database, which brings the schema changes of the earlier chapters into play. Use a procedure when server-side encapsulation or fewer round trips are valuable, especially when several applications must share the same database-side operation. A transaction makes a multi-step change succeed or fail together whether the steps live in the application or in a procedure, so atomicity alone is not a reason to write one. Keep everything else in the application, where it is easier to test and change.

Choosing Indexes

There is no formula, but there are habits that work:

- **Start from the queries, not the columns.** List the queries the application sends most often, and index for those.
- **Index what you search, join, and sort by.** Columns in `where`, `join ... on`, and `order by` are the candidates. Primary keys are indexed for you.
- **Put equality columns first, range columns last** in a multi-column index.
- **Drop an index that is the beginning of another.**
- **Read the plan, then measure.** An index that is never used costs space and slows every write for nothing.
- **Add covering columns for the few queries that run constantly**, not for all of them.
- **Remember the write cost.** A table that is mostly read can carry many indexes. A table that is mostly written cannot.
- **Run `analyze`** after loading a lot of data or creating indexes.

The IMDb Explorer Workbook

The workbook in `code/gh-codespaces-imdb-workbook.md` is an older, extended exercise that builds a Flask application on the IMDb data in a GitHub Codespace, with a search page, filters, a title page, and an analytics page. Its setup differs from this chapter. It loads four IMDb files, not two, and it uses Pandas instead of our standard-library loader. It also creates its secondary indexes in `schema.sql` before the import, where this chapter loads first and indexes afterward. Apply the ideas from this chapter to it, and expect the setup steps to differ. Its search uses `like '%term%'` and sorts by vote count, so it scans and sorts. Its filters combine a type and a range of years, which the two-column index suits. Before building any of it, run the plan for each query the application will send and decide which ones need an index.

Practice

Work in a copy of the code directory. Use Python 3.10 or newer. The scripts need only the standard library.

1. Run `python3 -m unittest -v`. The tests build a small IMDb-format file, so they need no download. Read the test names and find the ones about the plans.
2. Follow the California-orange lookup in the illustrations. Identify which rows each index lets SQLite skip.
3. Download the two IMDb files and run `import_imdb.py`. Run `ls -l imdb.db` and note the size.
4. Run `explore_indexes.py`. Which step gained the most from an index? Which gained the least? Which index cost the most space?
5. Run `compare_python_sql.py`. Change the number of titles from 2,000 to 20,000. Does the ratio between the two approaches change?
6. Build `idx_titles_type_year` yourself in `sqlite3` or Python, then write the plan for `where start_year = 1994`. Before `analyze`, why does it not search?
7. Run `analyze`; and compare the plans again, starting with `where start_year = 1994`. What changed, and what does the plan say about `title_type`?

Exercise: Tune Three Queries

Each of these queries runs slowly on `imdb.db` with only the primary key indexes. For each one, read the plan, predict what index would help and why, build it, and check the plan and the time. Then explain why your column order is right, and what the index cost.

1. Movies longer than 300 minutes, longest first: `select primary_title, runtime_minutes from titles where title_type = 'movie' and runtime_minutes > 300 order by runtime_minutes desc`
2. Highly rated titles with many votes: `select tconst, average_rating from ratings where average_rating >= 9.5 and num_votes >= 10000`
3. The average rating of movies by decade: `select t.start_year / 10 * 10, avg(r.average_rating) from titles t join ratings r on r.tconst = t.tconst where t.title_type = 'movie' and r.num_votes >= 100 group by 1`

The third one is slower than you might expect even with sensible indexes. Read its plan, count the work it does, and explain why an index does not make it fast.

References

Sources accessed September 24, 2026; SQLite query planning and plan documentation rechecked September 30, 2026.

1. SQLite. “Query Planning.” (Source of the figures reproduced in this chapter.) <https://www.sqlite.org/queryplanner.html>
2. Wikipedia contributors. “IMDb.” Wikipedia. <https://en.wikipedia.org/wiki/IMDb>
3. IMDb. “IMDb Non-Commercial Datasets.” <https://developer.imdb.com/non-commercial-datasets/>
4. SQLite. “SQLite Copyright: SQLite Is Public Domain.” Accessed September 30, 2026. <https://www.sqlite.org/copyright.html>
5. SQLite. “EXPLAIN QUERY PLAN.” <https://www.sqlite.org/eqp.html>
6. SQLite. “The SQLite Query Optimizer Overview.” <https://www.sqlite.org/optoverview.html>
7. Wikipedia contributors. “Full table scan.” Wikipedia. https://en.wikipedia.org/wiki/Full_table_scan
8. Wikipedia contributors. “Binary search algorithm.” Wikipedia. https://en.wikipedia.org/wiki/Binary_search_algorithm
9. Wikipedia contributors. “Big O notation.” Wikipedia. https://en.wikipedia.org/wiki/Big_O_notation
10. Wikipedia contributors. “Database index.” Wikipedia. https://en.wikipedia.org/wiki/Database_index
11. Wikipedia contributors. “B-tree.” Wikipedia. <https://en.wikipedia.org/wiki/B-tree>
12. Wikipedia contributors. “Query plan.” Wikipedia. https://en.wikipedia.org/wiki/Query_plan
13. Wikipedia contributors. “Query optimization.” Wikipedia. https://en.wikipedia.org/wiki/Query_optimization
14. SQLite. “CREATE INDEX.” https://www.sqlite.org/lang_createindex.html
15. SQLite. “ANALYZE.” https://www.sqlite.org/lang_analyze.html

16. Wikipedia contributors. “Tab-separated values.” Wikipedia. https://en.wikipedia.org/wiki/Tab-separated_values
17. Python Software Foundation. “sqlite3: DB-API 2.0 interface for SQLite databases.” Python documentation. <https://docs.python.org/3/library/sqlite3.html>
18. SQLite. “SQL Language Expressions,” section on the LIKE operator. https://www.sqlite.org/lang_expr.html
19. SQLite. “SQLite FTS5 Extension.” <https://www.sqlite.org/fts5.html>
20. Wikipedia contributors. “Stored procedure.” Wikipedia. https://en.wikipedia.org/wiki/Stored_procedure
21. Wikipedia contributors. “PostgreSQL.” Wikipedia. <https://en.wikipedia.org/wiki/PostgreSQL>
22. PostgreSQL Global Development Group. “CREATE PROCEDURE.” PostgreSQL documentation. <https://www.postgresql.org/docs/current/sql-createprocedure.html>

Questions for Thought

1. An index makes one query faster and every write slower. How would you decide whether an index on a column is worth having? What information would you need about the application?
2. The multi-column index on (`title_type`, `start_year`) helped queries on `start_year` when `title_type` was also given. It did not help queries on `start_year` alone until `analyze` made a skip-scan possible. What does that tell you about designing indexes from the queries an application will send, and about relying on an optimization that depends on statistics?
3. The covering index was a small improvement on a query that already used an index. When would it matter, and when would it be wasted space?
4. Counting movies by decade in Python and in SQL took about the same time with SQLite. Why? What would change with a database server on another machine?
5. A stored procedure can enforce a rule for every application that uses a database. What does that gain over enforcing the rule in the application, and what does it cost?
6. `like '%term%'` cannot use an ordinary index. How might a search feature be designed around that, and what would you give up?
7. Every timing in this chapter comes from one machine. What would you need to do before trusting a claim that an index made a query faster?